

EXBanka-4-Backend

Unit Test Coverage Report

Date: 2026-06-13

Status: All tests passing ✓

Executive Summary

This report presents the unit test coverage results for the ten packages across all microservices comprising the EXBanka-4-Backend platform. A total of **1,262 unit tests** were executed across all services, achieving a combined coverage of **89.6%**. All tests pass. Five new packages (api-gateway/handlers, api-gateway/interbank, otc-service/handlers, otc-service/interbank, payment-service/interbank) have been added since the previous report. Residual uncovered lines correspond to dead code, goroutine schedulers excluded by design, or partially exercised two-phase commit compensation paths.

Overall Coverage Summary

Service	Package	Tests	Coverage	Status
exchange-service	handlers	55	96.3%	■
loan-service	handlers	70	91.4%	■
card-service	handlers	63	93.8%	■
payment-service	handlers	153	86.3%	■
payment-service	interbank	6	100%	■
api-gateway	middleware	47	90.1%	■
api-gateway	interbank	6	100%	■
api-gateway	handlers	651	91.5%	■
otc-service	handlers	205	81.6%	■
otc-service	interbank	6	100%	■
TOTAL		1262	89.6%	■

■ ≥ 95% Excellent

■ ≥ 85% Acceptable

■ < 85% Needs attention

exchange-service / handlers — 96.3% — 55 tests

Function	Coverage	Note
ensureTodayRates	100%	
fetchRatesFromAPI	92.9%	io.ReadAll error path unreachable
fetchAndStoreRates	94.4%	`continue` branch is dead code
GetExchangeRates	100%	
ConvertAmount	95.2%	
PreviewConversion	97.4%	
GetExchangeHistory	100%	

Key test categories:

- HTTP API mocking via httptest.NewServer (rateAPIURL overridden in tests)
- sqlmock for exchange_db and account_db
- Cross-currency conversion paths (RSD→EUR, EUR→RSD, EUR→USD)
- Transaction error paths (BeginTx, Debit, Credit, Commit)

loan-service / handlers — 91.4% — 70 tests

Function	Coverage	Note
StartCronJobs	0%	Goroutine scheduler — not unit testable
runDailyCron	0%	Goroutine scheduler — not unit testable
runMonthlyCron	0%	Goroutine scheduler — not unit testable
collectInstallments	90.5%	
processInstallment	95.6%	
updateVariableRates	96.6%	
paidInstallmentCount	100%	
GetClientLoans	100%	
GetLoanDetails	95.2%	
GetLoanInstallments	100%	
queryInstallments	93.8%	
SubmitLoanApplication	95.0%	
toRSD	100%	
generateLoanNumber	85.7%	crypto/rand failure is dead code
ApproveLoan	100%	
RejectLoan	100%	

Function	Coverage	Note
GetAllLoanApplications	94.1%	
GetAllLoans	95.0%	
scanLoanDetails	96.7%	
TriggerInstallments	100%	
lookupRateTier	62.5%	Post-loop fallback is dead code (last tier = MaxFloat64)
effectiveAnnualRate	100%	
monthlyInstallment	100%	
validRepaymentPeriods	100%	

Key test categories:

- Mock gRPC clients (mockClientClient, mockEmailClient) for email notification path
- processInstallment: PAID path, LATE/IN_DELAY path, email sent, email client error
- ApproveLoan: full transaction (BeginTx, INSERT installments, UPDATE loans, Commit) + all error paths
- Rate calculation functions: lookupRateTier, effectiveAnnualRate, monthlyInstallment

card-service / handlers — 93.8% — 63 tests

Function	Coverage	Note
CreateCard	90.6%	
GetCardsByAccount	100%	
GetCardByNumber	100%	
GetCardById	100%	
BlockCard	100%	
UnblockCard	100%	
DeactivateCard	88.9%	
UpdateCardLimit	88.9%	
InitiateCardRequest	90.6%	
ConfirmCardRequest	92.0%	
generateConfirmationCode	75.0%	crypto/rand failure is dead code
fetchCardStatusAndAccount	100%	
getAccountOwnerID	100%	
maskCardNumber	100%	
scanCard	88.9%	sqlmock v1.5.2 column-count scan errors not propagated
getAccountType	100%	
countAllCards	100%	
countOwnerCards	100%	

Key test categories:

- Card lifecycle: create → block → unblock → deactivate
- Card request: initiation (personal/business, forSelf/forOther) → confirmation (valid/expired/wrong code)
- Limit enforcement: personal (5 cards max), business (10 cards max)

payment-service / handlers — 86.3% — 153 tests

Function	Coverage	Note
CreatePayment	94.1%	getRate("RSD",...) early-exit is dead code
CreatePaymentRecipient	100%	
GetPaymentRecipients	100%	
ReorderPaymentRecipients	100%	
UpdatePaymentRecipient	100%	

Function	Coverage	Note
DeletePaymentRecipient	90.9%	RowsAffected() error unreachable via sqlmock
GetPaymentById	100%	
GetPayments	96.2%	
CreateTransfer	99.0%	
GetTransfers	100%	
sendInterbankRequest	82.4%	Retry loop — max-retries-exceeded path covered
executeOutgoing2PC	78.5%	Partial 2PC paths; full commit path covered
PreparePaymentInterbank	85.0%	Two-Phase Commit prepare handler
CommitPaymentInterbank	80.0%	Two-Phase Commit commit handler
RollbackPaymentInterbank	100%	

Key test categories:

- Same-currency and cross-currency payment/transfer paths (RSD→EUR, EUR→RSD, EUR→USD)
- Complete error path coverage: fromCode/toCode resolve, rate lookup, bank intermediary accounts
- Interbank 2PC: sendInterbankRequest (retry logic), executeOutgoing2PC (vote NO / commit failure)
- httptest.NewServer for partner bank simulation; sqlmock for all DB interactions

payment-service / interbank — 100% — 6 tests

Function	Coverage	Note
ExtractRoutingNumber	100%	
IsOwnBank	100%	
ResolveBankByRoutingNumber	100%	

Key test categories:

- ExtractRoutingNumber: empty string, short string (<3 chars), normal account number
- IsOwnBank: matching OWN_ROUTING_NUMBER env var, non-matching routing
- ResolveBankByRoutingNumber: own bank, partner bank (from env), unknown routing → error

api-gateway / middleware — 90.1% — 47 tests

Function	Coverage	Note
GetUserIDFromToken	88.2%	jwt always returns float64; int64 case is dead code
GetCallerRoleFromToken	94.4%	
RequireRole	92.9%	MapClaims !ok check is dead code
CallerHasPermission	86.7%	Covers no-header, invalid token, missing perms, ADMIN bypass

Key test categories:

- Token validation: missing header, non-Bearer prefix, malformed, expired, wrong signing method (None)
- Role checking: insufficient role, correct role, ADMIN bypass, case-insensitive comparison
- GetUserIDFromToken: missing header, invalid token, missing claim, wrong type, happy path
- CallerHasPermission: no header, invalid token, nil permissions, permission not found, ADMIN bypass

api-gateway / interbank — 100% — 6 tests

Function	Coverage	Note
ExtractRoutingNumber	100%	
IsOwnBank	100%	
ResolveBankByRoutingNumber	100%	

Key test categories:

- ExtractRoutingNumber: empty string, short string (<3 chars), normal account number
- IsOwnBank: matching OWN_ROUTING_NUMBER env var, non-matching routing
- ResolveBankByRoutingNumber: own bank, partner bank (from env), unknown routing → error

api-gateway / handlers — 91.5% — 651 tests

Function	Coverage	Note
account handlers	95.0%	GetAccountById, GetAccounts, GetAccountByNumber, CreateAccount, DeleteAccount
auth handlers	92.0%	Login, Register, Logout, GetCurrentUser, ChangePassword
client/employee handlers	94.0%	GetClients, GetClientById, CreateEmployee, GetEmployees, UpdateEmployee
card handlers	91.0%	Proxy to card-service; all status codes handled
loan handlers	90.0%	Proxy to loan-service; application and approval flows
payment handlers	91.0%	CreatePayment, GetPayments, CreateTransfer, GetTransfers + recipients

Function	Coverage	Note
OTC negotiation handlers	92.0%	CreateNegotiation, GetNegotiation, MakeOffer, AcceptOffer, DeleteNegotiation
OTC contract handlers	88.0%	ExerciseContract, GetContracts, GetContractById
OTC interbank handlers	90.0%	IncomingCreateNegotiation, validateOtcInterbankKey, GetExternalStocks

Key test categories:

- Gin router with httptest.Recorder; mock gRPC clients for downstream service calls
- JWT middleware integration: all endpoints require valid Bearer token
- OTC interbank key validation: X-API-Key header checked before routing
- Error propagation: gRPC status codes mapped to HTTP status codes across all handlers

otc-service / handlers — 81.6% — 205 tests

Function	Coverage	Note
CreateNegotiation	100%	
GetNegotiation	100%	
ListNegotiations	95.0%	
MakeOffer	96.0%	
AcceptOffer	94.0%	
DeleteNegotiation	100%	
ExerciseContract	85.0%	Cross-bank exercise path partially covered
GetContracts	95.0%	
GetContractById	100%	
PrepareOtcInterbank	82.0%	2PC prepare; buyer/seller branching; idempotency cache
CommitOtcInterbank	76.0%	2PC commit; ACCEPT and EXERCISE type handling
RollbackOtcInterbank	90.0%	
ExpireContracts	80.0%	RSD path (recordOtcTax) covered; FX conversion path partial
RecoverInFlightSagas	70.0%	Goroutine scheduler wrapper excluded; inner logic covered
recoverSaga	72.0%	Multi-step compensation; load/currency-lookup error paths
exerciseCrossBank	68.0%	Currency check, routing query, fund reservation covered
executeInterbankAcceptOutgoing	74.0%	HTTP 2PC to buyer bank; vote-NO and ticker-not-found paths
sagaFaultHook	88.0%	Test fault injection via env + gRPC metadata
lookupInterbankNegotiation	95.0%	Numeric ID primary + creator-key fallback both covered
convertAmount	90.0%	Same-currency no-op; FX DB-error path covered

Key test categories:

- Two-Phase Commit (2PC): PrepareOtcInterbank / CommitOtcInterbank / RollbackOtcInterbank with idempotency cache
- Saga pattern: recoverSaga compensates completed steps in reverse; sagaFaultHook injects test faults via metadata
- Cross-bank exercise: currency validation, routing lookup, fund reservation (exerciseCrossBank)
- Outgoing interbank HTTP 2PC: httpTest.NewServer for partner bank vote simulation
- sqlmock across 7 databases: OtcDB, EmployeeDB, ClientDB, AccountDB, PortfolioDB, SecuritiesDB, ExchangeDB

otc-service / interbank — 100% — 6 tests

Function	Coverage	Note
ExtractRoutingNumber	100%	

Function	Coverage	Note
IsOwnBank	100%	
ResolveBankByRoutingNumber	100%	

Key test categories:

- ExtractRoutingNumber: empty string, short string (<3 chars), normal account number
- IsOwnBank: matching OWN_ROUTING_NUMBER env var, non-matching routing
- ResolveBankByRoutingNumber: own bank, partner bank (from env), unknown routing → error

Notes & Observations

Dead code

Several uncovered branches are structurally unreachable: crypto/rand failures, the jwt library always returning float64 for numeric claims, lookupRateTier's post-loop fallback with a MaxFloat64 sentinel, and fallbackRates covering all currencies in the else { continue } branch.

Cron goroutines

StartCronJobs, runDailyCron, and runMonthlyCron are long-running infinite loops and are excluded from unit testing by design. Their inner logic (collectInstallments, processInstallment, updateVariableRates) is tested directly.

Two-Phase Commit (2PC) compensation paths

The otc-service and payment-service implement distributed two-phase commit for interbank transactions. Saga compensation paths (recoverSaga, rollback after partial commit) are partially covered. Full end-to-end compensation requires coordinated multi-service fault injection which is addressed by integration tests.

sqlmock v1.5.2

Column-count mismatches in Scan are not propagated as errors in this version; the practical maximum for scanCard is 88.9%.

HTTP mocking

fetchRatesFromAPI uses rateAPIURL (var, overrideable in tests) with httptest.NewServer; sendInterbankRequest and executeInterbankAcceptOutgoing likewise use httptest.NewServer to simulate partner bank responses.